

Edge-Linking-using-Hough-Transform ¶

Name : Iniya E

Reg No : 212224230096

```
In [1]: import numpy as np
import cv2
import matplotlib.pyplot as plt

# READ THE IMAGE IN COLOR
img_c = cv2.imread("wonders.jpg")
# CONVERT THE COLOR FROM BGR TO RGB
img_c = cv2.cvtColor(img_c, cv2.COLOR_BGR2RGB)
# CONVERT THE COLOR IMAGE TO GRAYSCALE
gray = cv2.cvtColor(img_c, cv2.COLOR_RGB2GRAY)
# APPLY GAUSSIAN BLUR TO REDUCE NOISE
gray = cv2.GaussianBlur(gray, (3, 3), 0)
```

```
In [2]: # DISPLAY ORIGINAL AND GRAY IMAGES
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.imshow(img_c)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(gray, cmap='gray')
plt.title("Gray Image")
plt.axis("off")
plt.show()
```

Original Image



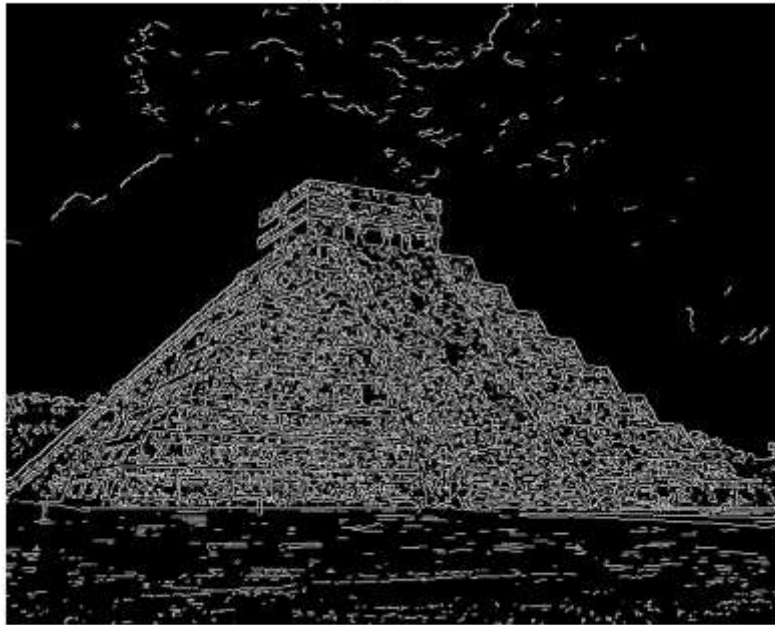
Gray Image



```
In [3]: # CANNY EDGE DETECTION
canny = cv2.Canny(gray, 120, 150)

# DISPLAY THE CANNY IMAGE
plt.figure(figsize=(5, 8))
plt.imshow(canny, cmap='gray')
plt.title("Canny Edge Detector")
plt.axis("off")
plt.show()
```

Canny Edge Detector



```
In [4]: # HOUGH LINE TRANSFORM
lines = cv2.HoughLinesP(canny, 1, np.pi / 180, threshold=80, minLineLength=50,

# DRAW THE DETECTED LINES ON THE ORIGINAL IMAGE
for line in lines:
    x1, y1, x2, y2 = line[0]
    cv2.line(img_c, (x1, y1), (x2, y2), (255, 0, 0), 3) # Red color Lines
```

```
In [5]: # DISPLAY THE RESULT IMAGE WITH LINES
plt.figure(figsize=(5, 8))
plt.imshow(img_c)
plt.title("Result Image")
plt.axis("off")
plt.show()
```

Result Image

